INTERVIEW PREP SERIES

SQL & Queries

Interview Questions & Easy Answers

---

52 questions · 25 marked ★ most asked

Covers SQL basics, keys and constraints, joins, GROUP BY/HAVING, subqueries, window functions, normalization, ACID and transactions, indexes, plus the most-asked query problems (second highest salary, duplicates, Nth salary, department-wise max). Queries use two sample tables: `Employee(id, name, salary, dept_id, manager_id, hire_date)` and `Department(id, dept_name)`.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Basics and concepts

Q1. What is SQL? What is the difference between SQL and MySQL? ★ Most asked

SQL (Structured Query Language) is the standard language used to store, read, update and delete data in relational databases. MySQL is a **database management system** (software) that uses SQL. Others include PostgreSQL, Oracle and SQL Server.

Q2. What are DDL, DML, DCL and TCL commands? ★ Most asked

- **DDL** (Data Definition): `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`, which define structure.
- **DML** (Data Manipulation): `SELECT`, `INSERT`, `UPDATE`, `DELETE`, which work on data.
- **DCL** (Data Control): `GRANT`, `REVOKE`, which manage permissions.
- **TCL** (Transaction Control): `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

Q3. What is the difference between DELETE, TRUNCATE and DROP? ★ Most asked

- `DELETE` removes selected rows (can use `WHERE`), is DML, logs each row, can be rolled back, and fires triggers.
- `TRUNCATE` removes all rows quickly, is DDL, keeps the table structure, resets auto-increment, and cannot use `WHERE`.
- `DROP` deletes the whole table, structure and data.

Q4. What are the different keys in a database? ★ Most asked

- **Primary key**: uniquely identifies each row; cannot be NULL; one per table.
- **Foreign key**: a column that refers to the primary key of another table; links tables.
- **Unique key**: values must be unique, but one NULL is usually allowed.
- **Candidate key**: any column(s) that could be the primary key.
- **Composite key**: a primary key made of two or more columns.
- **Super key**: any set of columns that uniquely identifies a row.

Q5. What is the difference between a primary key and a unique key? ★ Most asked

Both enforce uniqueness. A primary key cannot be NULL and there's only one per table. A unique key can accept a NULL (in most databases) and a table can have many unique keys.

Q6. What are constraints?

Rules on columns that keep data valid: `NOT NULL`, `UNIQUE`, `PRIMARY KEY`, `FOREIGN KEY`, `CHECK` (e.g. `salary > 0`) and `DEFAULT`.

Q7. What is the difference between `CHAR` and `VARCHAR`?

`CHAR(10)` always uses 10 characters of space (padded with spaces), so it suits fixed-length data like codes. `VARCHAR(10)` uses only as much space as the actual text, up to 10.

Q8. What is the order of execution of a SQL query? ★ Most asked

`FROM` / `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`.

This is why you can't use a `SELECT` alias inside `WHERE` (WHERE runs before SELECT).

Filtering, grouping and joins

Q9. What is the difference between WHERE and HAVING? ★ Most asked

`WHERE` filters rows **before** grouping and can't use aggregate functions. `HAVING` filters groups **after** `GROUP BY` and can use aggregates.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM Employee
WHERE salary > 10000
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

Q10. What are aggregate functions? ★ Most asked

Functions that calculate one value from many rows: `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`. `COUNT(*)` counts all rows; `COUNT(column)` ignores NULLs.

Q11. What are the types of JOINs? ★ Most asked

- **INNER JOIN**: only rows that match in both tables.
- **LEFT JOIN**: all rows from the left table + matches from the right (NULL if none).
- **RIGHT JOIN**: all rows from the right table + matches from the left.
- **FULL OUTER JOIN**: all rows from both, with NULLs where there's no match.
- **CROSS JOIN**: every row of one table with every row of the other (Cartesian product).
- **SELF JOIN**: a table joined with itself.

```
SELECT e.name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.id;
```

Q12. Write a query to show each employee with their manager's name (self join). ★ Most asked

```
SELECT e.name AS employee, m.name AS manager
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.id;
```

LEFT JOIN keeps employees with no manager (like the CEO).

Q13. What is the difference between UNION and UNION ALL?

Both combine results of two SELECTs with the same columns. `UNION` removes duplicates (slower). `UNION ALL` keeps all rows including duplicates (faster).

Q14. What is the difference between `IN`, `BETWEEN` and `LIKE`?

`IN` matches any value in a list (`dept_id IN (1,2)`). `BETWEEN` matches a range, inclusive (`salary BETWEEN 30000 AND 50000`). `LIKE` matches patterns: `%` = any characters, `_` = one character (`name LIKE 'A%'`).

Q15. How do you handle NULL values?

Use `IS NULL` / `IS NOT NULL` (not `= NULL`). Replace NULL with a default using `COALESCE(col, 0)` or `IFNULL` (MySQL). Any arithmetic with NULL gives NULL.

Most asked query problems

Q16. Find the second highest salary. ★ Most asked

```
-- Method 1: subquery
SELECT MAX(salary) AS second_highest
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Method 2: LIMIT / OFFSET (MySQL, PostgreSQL)
SELECT DISTINCT salary FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Interview tip: Asked in TCS, UKG and many other reported interviews. Know at least two methods and mention DISTINCT for duplicate salaries.

Q17. Find the Nth highest salary. ★ Most asked

```
-- Using DENSE_RANK (works for any N; here N = 3)
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM Employee
) t
WHERE rnk = 3
LIMIT 1;

-- Using LIMIT/OFFSET: OFFSET N-1
SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

Q18. Find the highest salary in each department. ★ Most asked

```
SELECT d.dept_name, MAX(e.salary) AS max_salary
FROM Employee e
JOIN Department d ON e.dept_id = d.id
GROUP BY d.dept_name;
```

To show the employee name too, use a window function:

```
SELECT name, dept_id, salary FROM (
  SELECT name, dept_id, salary,
         RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM Employee
) t WHERE rnk = 1;
```

Q19. Find duplicate records in a table. ★ Most asked

```
SELECT name, COUNT(*) AS cnt
FROM Employee
GROUP BY name
HAVING COUNT(*) > 1;
```

Q20. Delete duplicate rows but keep one. ★ Most asked

```
-- MySQL
DELETE e1 FROM Employee e1
JOIN Employee e2
  ON e1.name = e2.name AND e1.id > e2.id;
```

```
-- Standard SQL using ROW_NUMBER
WITH cte AS (
  SELECT id, ROW_NUMBER() OVER (PARTITION BY name ORDER BY id) AS rn
  FROM Employee
)
DELETE FROM Employee WHERE id IN (SELECT id FROM cte WHERE rn > 1);
```

Q21. Find employees who earn more than their manager. ★ Most asked

```
SELECT e.name
FROM Employee e
JOIN Employee m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Q22. Find employees whose salary is above the average salary. ★ Most asked

```
SELECT name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

Q23. Find employees who earn more than the average of their own department.

```
SELECT e.name, e.salary, e.dept_id
FROM Employee e
WHERE e.salary > (
  SELECT AVG(salary) FROM Employee WHERE dept_id = e.dept_id
);
```

This is a **correlated subquery**: the inner query runs for each row of the outer query.

Q24. Count employees in each department, including departments with zero employees. ★ Most asked

```
SELECT d.dept_name, COUNT(e.id) AS total
FROM Department d
LEFT JOIN Employee e ON e.dept_id = d.id
GROUP BY d.dept_name;
```

`COUNT(e.id)` gives 0 for empty departments, while `COUNT(*)` would give 1.

Q25. Find departments with no employees.

```
SELECT d.dept_name
FROM Department d
LEFT JOIN Employee e ON e.dept_id = d.id
WHERE e.id IS NULL;
```

Q26. Find the top 3 highest-paid employees.

```
SELECT name, salary FROM Employee ORDER BY salary DESC LIMIT 3;
-- SQL Server: SELECT TOP 3 name, salary FROM Employee ORDER BY salary DESC;
```

Q27. Find employees whose name starts with 'A' and ends with 'a'.

```
SELECT * FROM Employee WHERE name LIKE 'A%a';
```

Q28. Fetch employees who joined in the last 6 months.

```
SELECT * FROM Employee
WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);  -- MySQL
```

```
-- PostgreSQL: WHERE hire_date >= CURRENT_DATE - INTERVAL '6 months'
```

Q29. Fetch odd or even rows from a table.

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (ORDER BY id) AS rn FROM Employee
) t WHERE rn % 2 = 0;   -- even rows; use = 1 for odd
```

Q30. Increase the salary of all employees in the IT department by 10%.

```
UPDATE Employee
SET salary = salary * 1.10
WHERE dept_id = (SELECT id FROM Department WHERE dept_name = 'IT');
```

Q31. Show a category column using CASE.

```
SELECT name, salary,
  CASE
    WHEN salary >= 80000 THEN 'High'
    WHEN salary >= 40000 THEN 'Medium'
    ELSE 'Low'
  END AS band
FROM Employee;
```

Subqueries, views and window functions

Q32. What is a subquery? What is a correlated subquery? ★ Most asked

A subquery is a query inside another query, usually in `WHERE`, `FROM` or `SELECT`. A normal subquery runs once. A **correlated** subquery refers to a column of the outer query, so it runs once per outer row (slower, but powerful).

Q33. What is the difference between RANK, DENSE_RANK and ROW_NUMBER? ★ Most asked

For salaries 100, 90, 90, 80:

- `ROW_NUMBER()`: 1, 2, 3, 4 (always unique)
- `RANK()`: 1, 2, 2, 4 (skips after a tie)
- `DENSE_RANK()`: 1, 2, 2, 3 (no gaps)

That's why `DENSE_RANK` is best for "Nth highest salary".

Q34. What are window functions?

Functions that calculate across a set of rows related to the current row **without collapsing them** like GROUP BY does. Syntax: `function() OVER (PARTITION BY ... ORDER BY ...)`. Examples: ranking, running totals (`SUM(salary) OVER (ORDER BY hire_date)`), `LAG()` and `LEAD()` to compare with the previous/next row.

Q35. What is a CTE (WITH clause)?

A Common Table Expression is a named temporary result you define at the start of a query to make it readable. Recursive CTEs can walk hierarchies like an org chart.

```
WITH high_paid AS (SELECT * FROM Employee WHERE salary > 70000)
SELECT dept_id, COUNT(*) FROM high_paid GROUP BY dept_id;
```

Q36. What is a view? ★ Most asked

A view is a saved SELECT query that acts like a virtual table. It doesn't store data itself (unless it's a materialized view). Uses: simplify complex queries, and security (show only certain columns to some users).

```
CREATE VIEW it_staff AS
SELECT name, salary FROM Employee WHERE dept_id = 2;
```

Q37. What are stored procedures and triggers?

A **stored procedure** is a saved set of SQL statements that you call by name with parameters (`CALL give_bonus(5)`). A **trigger** runs automatically when an event like INSERT, UPDATE or DELETE happens on a table, e.g. writing an audit log.

Normalization and design

Q38. What is normalization? Explain 1NF, 2NF and 3NF. ★ Most asked

Normalization organises tables to reduce duplicate data and avoid update problems (anomalies).

- **1NF**: every column holds atomic (single) values, with no repeating groups. Example: no "phone1, phone2" or comma lists in one cell.
- **2NF**: in 1NF, and every non-key column depends on the **whole** primary key (no partial dependency). Matters for composite keys.
- **3NF**: in 2NF, and no non-key column depends on another non-key column (no transitive dependency). Example: store `dept_name` in a Department table, not in Employee next to `dept_id`.
- **BCNF**: a stricter 3NF where every determinant is a candidate key.

Interview tip: TCS reported asking "Explain 3NF". Give the Employee/Department example.

Q39. What is denormalization?

Intentionally adding some duplicate data to reduce joins and speed up reads, common in reporting and high-traffic systems. It trades extra storage and update effort for faster queries.

Q40. What are anomalies in a non-normalized table?

- **Insertion anomaly**: can't add data without other unrelated data.
- **Update anomaly**: the same data stored in many rows must be updated everywhere.
- **Deletion anomaly**: deleting one row loses other important information.

Q41. What is an ER diagram?

An Entity-Relationship diagram shows entities (tables like Student, Course), their attributes (columns), and relationships (one-to-one, one-to-many, many-to-many). Many-to-many relationships become a separate junction table, like `Enrollment(student_id, course_id)`.

Transactions, indexes and performance

Q42. What are ACID properties? ★ Most asked

- **Atomicity**: all steps of a transaction happen, or none do. (Money transfer: debit and credit both happen or both fail.)
- **Consistency**: the database moves from one valid state to another, keeping all rules.
- **Isolation**: transactions running at the same time don't interfere with each other.
- **Durability**: once committed, data survives crashes (written to disk/logs).

Q43. What is a transaction? Show COMMIT and ROLLBACK.

A transaction is a group of SQL statements treated as one unit.

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;   -- or ROLLBACK; if something failed
```

Q44. What is an index? What are clustered and non-clustered indexes? ★ Most asked

An index is a data structure (usually a B-tree) that speeds up searching, like a book's index.

- **Clustered index**: decides the physical order of rows in the table; only one per table (usually the primary key).
- **Non-clustered index**: a separate structure with pointers to rows; a table can have many.

Indexes make reads faster but writes slightly slower and use extra space.

```
CREATE INDEX idx_emp_name ON Employee(name);
```

Q45. How do you optimise a slow query?

Add indexes on columns used in WHERE, JOIN and ORDER BY; select only needed columns instead of `SELECT *`; avoid functions on indexed columns in WHERE; use EXPLAIN to see the plan; avoid unnecessary subqueries; and use LIMIT with pagination.

Q46. What are isolation levels?

They control how much one transaction sees of others: **Read Uncommitted** (dirty reads possible), **Read Committed**, **Repeatable Read** (MySQL default), and **Serializable** (strictest, slowest). Higher levels prevent dirty reads, non-repeatable reads and phantom reads.

Q47. What is SQL injection and how do you prevent it?

An attack where a user enters SQL code in an input (like `' OR '1'='1`) to change your query. Prevent it with **parameterised queries / prepared statements**, ORMs, input validation and least-privilege DB users. Never build queries by joining strings with user input.

More frequently asked

Q48. What is the difference between a primary key and a foreign key?

A primary key uniquely identifies each row in its own table. A foreign key is a column in another table that points to that primary key, creating a link and enforcing that the referenced row exists (referential integrity). Example: `Employee.dept_id` is a foreign key to `Department.id`.

Q49. What are the advantages of a DBMS over a file system?

Less data redundancy, data consistency, easy querying with SQL, multi-user concurrent access, security and permissions, backup and recovery, and integrity rules through constraints.

Q50. What is the difference between `COUNT(*)`, `COUNT(1)` and `COUNT(column)`?

`COUNT(*)` and `COUNT(1)` both count all rows (same speed in modern databases). `COUNT(column)` counts only rows where that column is not NULL. `COUNT(DISTINCT column)` counts unique non-NULL values.

Q51. What is the difference between OLTP and OLAP?

OLTP (online transaction processing) handles many small, fast reads and writes for daily operations, like orders and payments. OLAP (online analytical processing) runs big, complex read queries over historical data for reports and dashboards.

Q52. Write a query to find the employee(s) with the highest salary in the whole company. ★

Most asked

```
SELECT name, salary
FROM Employee
WHERE salary = (SELECT MAX(salary) FROM Employee);
```

This returns all employees tied for the top salary, unlike `ORDER BY ... LIMIT 1`.